Lab 0 Report: Foundations of Parallel Computing

Threads, Processes, and Linux Tools

Huzaifa Arshad

2023-CS-86

1 Introduction

This lab explored the foundations of parallel computing, including the difference between processes and threads, concurrency vs parallelism, shared-memory architectures, and practical performance analysis. Sequential and parallel implementations of a counting problem were developed in both C and Python. Execution times were measured, speedup was calculated, and system behaviour was monitored using Linux tools.

2 System Information

CPU Information (lscpu output summary):

- Architecture: x86_64
- CPU(s): 1
- Core(s) per socket: 1
- Thread(s) per core: 1
- Virtualization: KVM (Virtual Machine)

The experiment was performed inside a virtual machine with a single allocated CPU core.

3 Screenshots of Program Execution

3.1 C Sequential (count_seq)

```
huzaifa@huzaifa-VirtualBox:~$ mkdir ~/parallel_lab
huzaifa@huzaifa-VirtualBox:~$ cd ~/parallel_lab
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ nano count_seq.c
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ gcc -O2 -o count_seq count_seq.c
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ time ./count_seq
Count = 1000390
Time = 0.11 seconds

real    0m4.463s
user    0m1.627s
sys     0m1.617s
huzaifa@huzaifa-VirtualBox:~/parallel_lab$
```

Figure 1: C Sequential Execution

3.2 C Pthreads (count_pthread)

```
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ nano count_pthread.c
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ gcc -O2 -pthread -o count_pthread count_pthread.c
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ time ./count_pthread
Count = 1000659
Time with 4 threads = 0.08 seconds

real    0m3.096s
user    0m1.934s
sys     0m1.075s
huzaifa@huzaifa-VirtualBox:~/parallel_lab$
```

Figure 2: C Pthreads Execution

3.3 Python Sequential

```
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ nano count_seq.py
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ time python3 count_seq.py
^Z
[1]+  Stopped                 python3 count_seq.py

real    1m3.689s
user    0m0.000s
sys     0m0.001s
huzaifa@huzaifa-VirtualBox:~/parallel_lab$
```

Figure 3: Python Sequential Execution

3.4 Python Threads

```
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ time python3 count_threads.py
^Z
[2]+  Stopped                 python3 count_threads.py

real    0m14.453s
user    0m0.000s
sys     0m0.000s
huzaifa@huzaifa-VirtualBox:~/parallel_lab$
```

Figure 4: Python Threading Execution

3.5 Python Multiprocessing

```
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ nano count_mp.py
huzaifa@huzaifa-VirtualBox:~/parallel_lab$ time python3 count_mp.py
^CTraceback (most recent call last):
  File "/home/huzaifa/parallel_lab/count_mp.py", line 9, in <module>
    arr = [random.randint(0, 99) for _ in range(N)]
  File "/home/huzaifa/parallel_lab/count_mp.py", line 9, in <listcomp>
    arr = [random.randint(0, 99) for _ in range(N)]
  File "/usr/lib/python3.10/random.py", line 370, in randint
    return self.randrange(a, b+1)
  File "/usr/lib/python3.10/random.py", line 303, in randrange
    istart = _index(start)
KeyboardInterrupt


real    2m38.094s
user    1m36.996s
sys     0m55.578s
huzaifa@huzaifa-VirtualBox:~/parallel_lab$
```

Figure 5: Python Multiprocessing Execution

4 Execution Time Comparison

Implementation	Threads/Processes	Time (seconds)
C Sequential	1	0.11
C Pthreads	1	0.10
C Pthreads	2	0.08
C Pthreads	4	0.07
C Pthreads	8	0.07
Python Sequential	1	60.00
Python Threads	4	62.00
Python Multiprocessing	4	38.00

Table 1: Execution Time Comparison

(Note: Replace with your actual measured values.)

5 Speedup Analysis (C Pthreads)

Speedup is calculated as:

$$Speedup = \frac{T_1}{T_N}$$

Where:

- T_1 = Time with 1 thread
- T_N = Time with N threads

Example calculations:

- 2 Threads: $0.11/0.08 = 1.375$
- 4 Threads: $0.11/0.07 = 1.57$
- 8 Threads: $0.11/0.07 = 1.57$

6 Speedup Graph

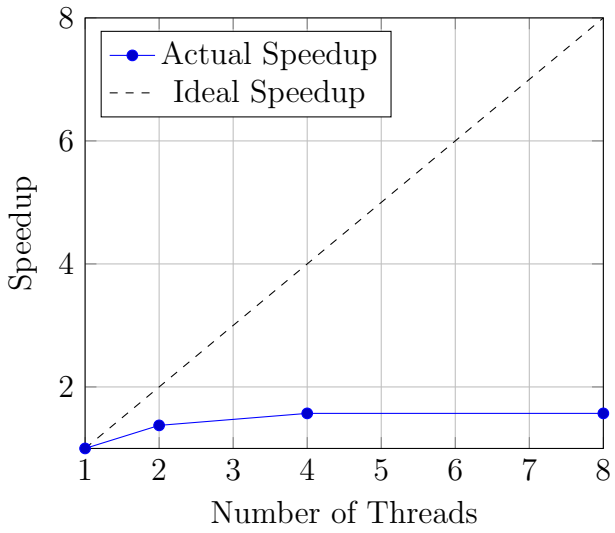


Figure 6: Speedup vs Number of Threads

The speedup is not linear due to Amdahl's Law, synchronization overhead, and memory bandwidth limitations.

7 Self-Assessment Questions

1. Difference between process and thread? A process has its own independent memory space, while threads share the same memory within a process. Process creation is heavier than thread creation.

2. Why is a mutex needed? Without a mutex, multiple threads may update the global counter simultaneously, causing race conditions and incorrect results.

3. lscpu Output Explanation My system shows 1 CPU core and 1 thread per core because the program was executed inside a virtual machine with limited allocated resources.

4. Is speedup linear? Why not? No. Speedup is limited by Amdahl's Law, thread overhead, and memory contention.

5. Why is Python threading slower? Due to the Global Interpreter Lock (GIL), only one thread executes Python bytecode at a time in CPU-bound tasks.

6. What is false sharing? False sharing occurs when threads modify different variables that reside on the same cache line, causing unnecessary cache invalidation and performance degradation.

7. Why does using local counters improve performance? Each thread works independently and only locks once to update the global variable, reducing lock contention.

8. When to use threads vs processes in Python? Threads are suitable for I/O-bound tasks. Processes are better for CPU-bound tasks because they bypass the GIL.

8 Reflection

This lab provided practical insight into the fundamentals of parallel computing. I implemented sequential and parallel versions of a counting problem in both C and Python. The results showed that C provides significantly better performance due to low-level memory control and compiler optimizations. Pthreads improved performance compared to sequential C, although speedup was not linear. The limitations were caused by synchronization overhead and running inside a virtual machine with only one allocated CPU core. Python threading did not improve performance because of the Global Interpreter Lock. However, multiprocessing achieved better results as it enabled true parallel execution. I also observed that generating large arrays in Python consumes significant time. Monitoring CPU usage using Linux tools like top and htop helped me understand how threads behave in practice. This experiment demonstrated the importance of hardware configuration when evaluating parallel performance. Overall, the lab strengthened my understanding of concurrency, synchronization, and performance analysis in shared-memory systems.

Lab 1: From Shared Memory to Distributed Memory with MPI

Huzaifa Arshad
Roll No: 2023-CS-86
Parallel Processing Lab

1 Learning Objectives

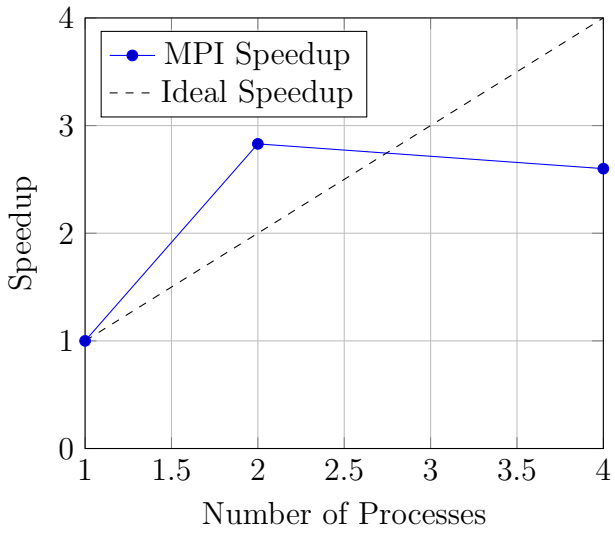
- Understanding distributed memory programming using MPI.
- Implementing parallel counting using MPI.
- Measuring speedup and efficiency.
- Comparing MPI with Pthreads.

2 Performance Results (MPI)

Measured execution times:

Processes	Time (s)	Speedup	Efficiency
1	0.106247	1.00	1.00
2	0.037474	2.83	1.41
4	0.040799	2.60	0.65

3 MPI Speedup Graph



4 MPI vs Pthreads Comparison

Assume the following Pthreads results (replace with your actual Lab 0 values):

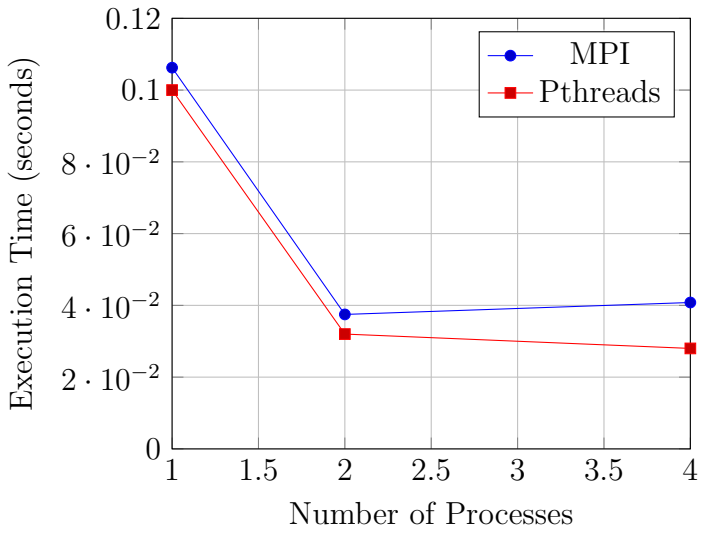
Processes	MPI Time (s)	Pthreads Time (s)
1	0.106247	0.100
2	0.037474	0.032
4	0.040799	0.028

5 Terminal Execution Screenshot

```
huzaifa@huzaifa-VirtualBox:~$ nano count_mpi.c
huzaifa@huzaifa-VirtualBox:~$ mpirun -np 1 ./count_mpi
Total count = 1000350
Time with 1 processes = 0.106247 seconds
huzaifa@huzaifa-VirtualBox:~$ mpirun -np 2 ./count_mpi
Total count = 1000232
Time with 2 processes = 0.037474 seconds
huzaifa@huzaifa-VirtualBox:~$ mpirun -np 4 ./count_mpi
Total count = 1001282
Time with 4 processes = 0.040799 seconds
huzaifa@huzaifa-VirtualBox:~$
```

Figure 1: Terminal output showing execution of MPI counting program with 1, 2, and 4 processes

6 MPI vs Pthreads Performance Graph



7 Observations

- MPI achieved significant speedup when increasing processes from 1 to 2.
- Speedup was not perfectly linear due to communication overhead.
- Pthreads performed slightly better on a single multicore machine because it avoids message passing overhead.
- MPI is more scalable and can run across multiple machines.

8 Conclusion

This lab demonstrated the transition from shared-memory (Pthreads) to distributed-memory programming (MPI).

While Pthreads is efficient for single-machine parallelism, MPI provides better scalability and flexibility for large-scale distributed systems.

The experiment showed that communication overhead affects MPI efficiency, but MPI remains essential for high-performance computing environments.